



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

CS6903: Network Security

(Module 4: TLS/SSL)

Saurabh Kumar
CSE Department, IIT Hyderabad

Acknowledgement

- ❑ Instructors from previous course floating
 - Prof. Bheemarjuna Reddy Tamma, Prof. Kotaro Kataoka, Prof. Antony Franklin
- ❑ Professors from other Universities/Insitutes
 - Sandeep K. Shukla, Stefan Savage, Dan Boneh, Kirill Levchenko, Alex Gantman, Deian Stefan, Nadia Heninger, Alex Dent, Vitaly Shamtikov, Robert Turner
- ❑ Other internet sources

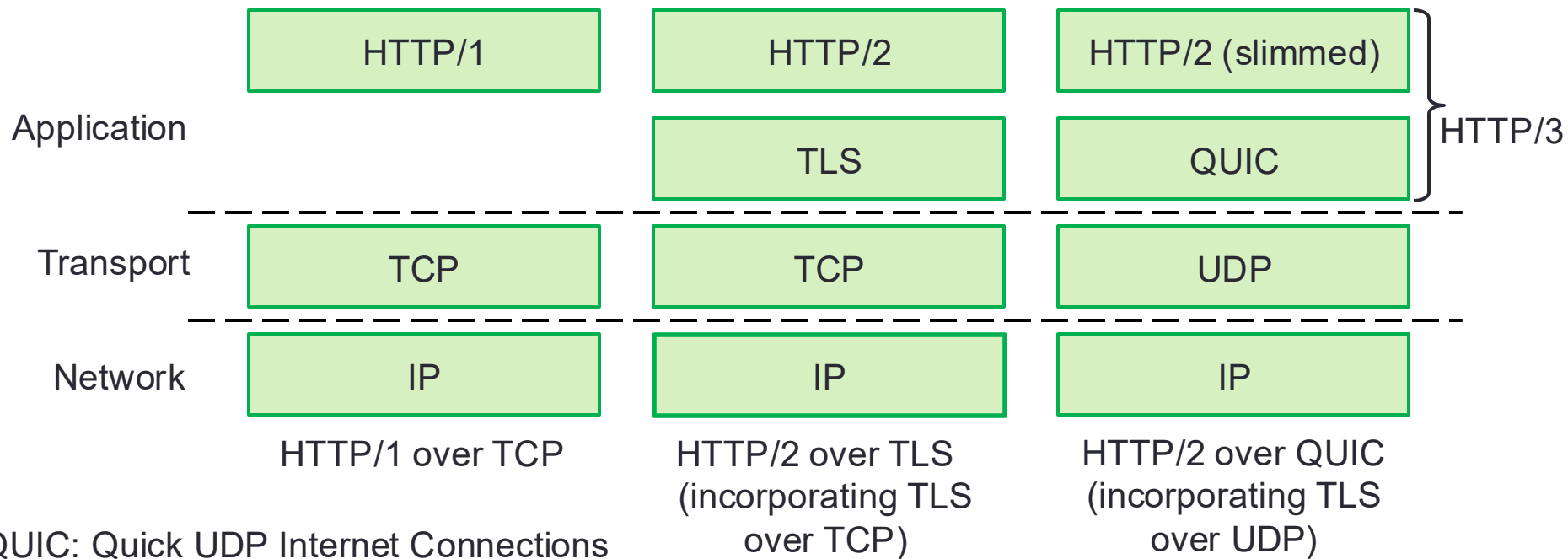
Transport Layer Security (TLS)

- ❑ The widely deployed security protocol above the transport layer
 - Supported by almost all browsers, web servers: https (port 443)
 - Primarily used with TCP (reliability and in-sequence delivery)
 - Datagram TLS (DTLS) variant for use with UDP/SCTP/SRTP/CAPWAP
- ❑ Provides:
 - **Confidentiality**: via symmetric encryption
 - **Integrity**: via cryptographic hashing
 - **Authentication**: via public key cryptography

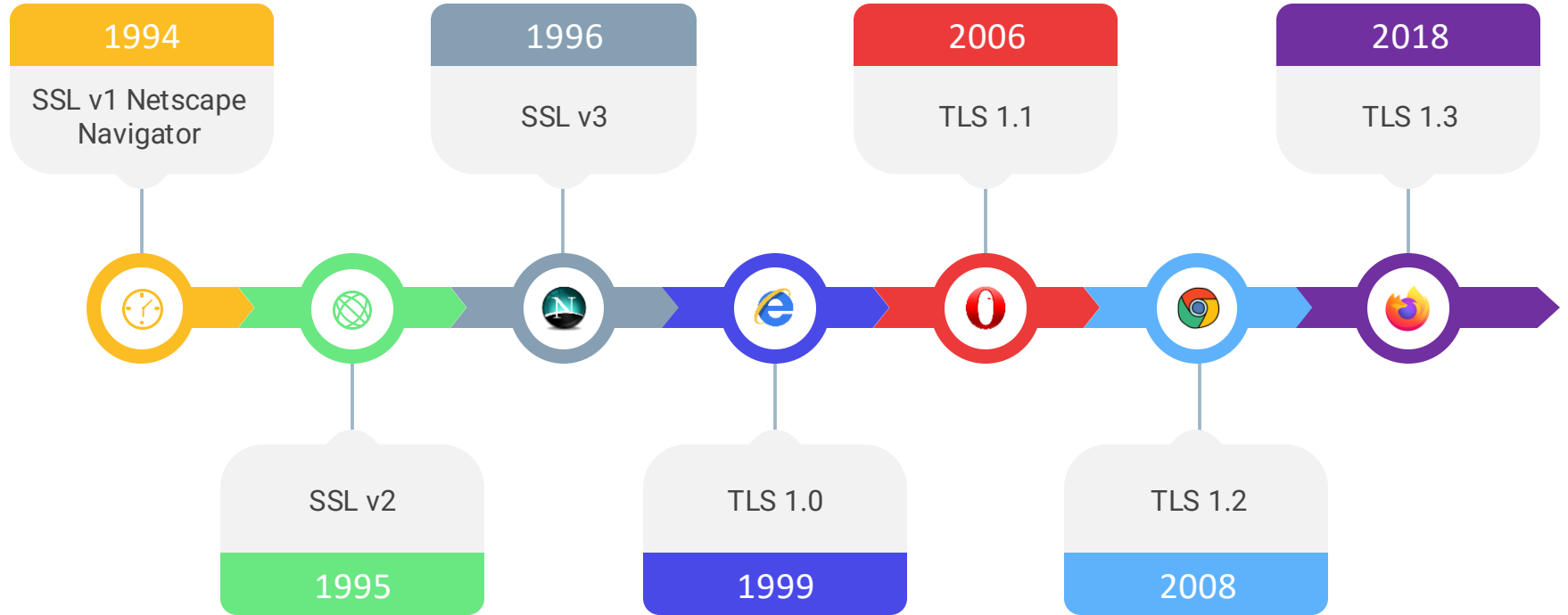
We have studied
all techniques!!

Transport Layer Security (TLS)

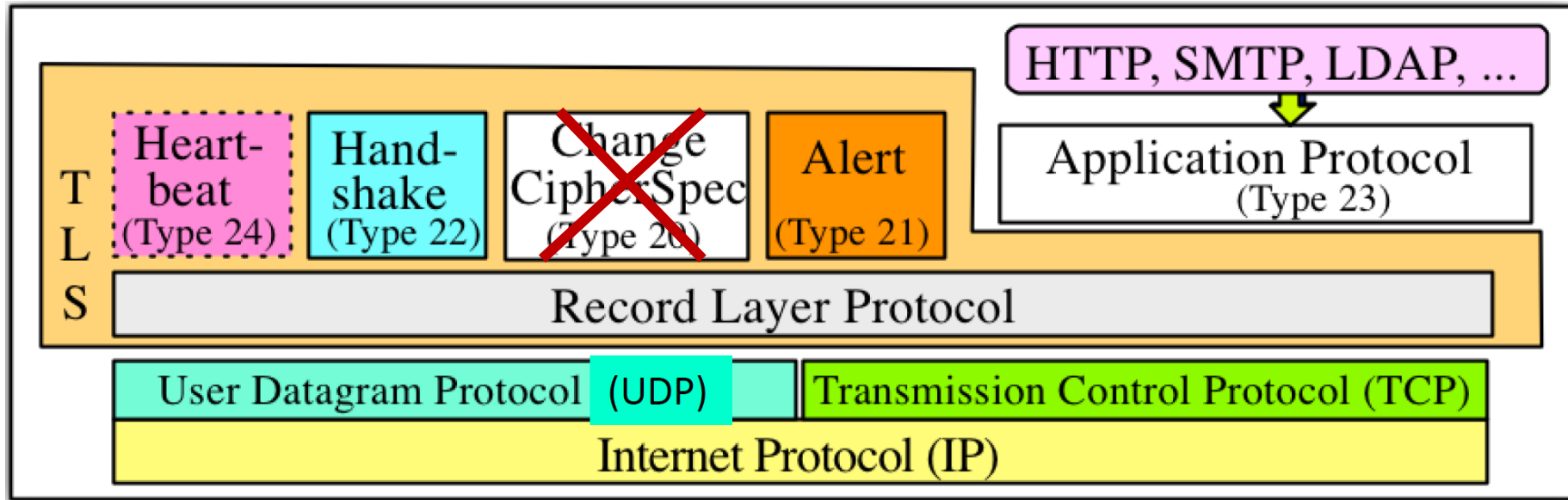
- ❑ TLS provides an API that any application can use
- ❑ HTTP view of TLS:



SSL/TLS Variants



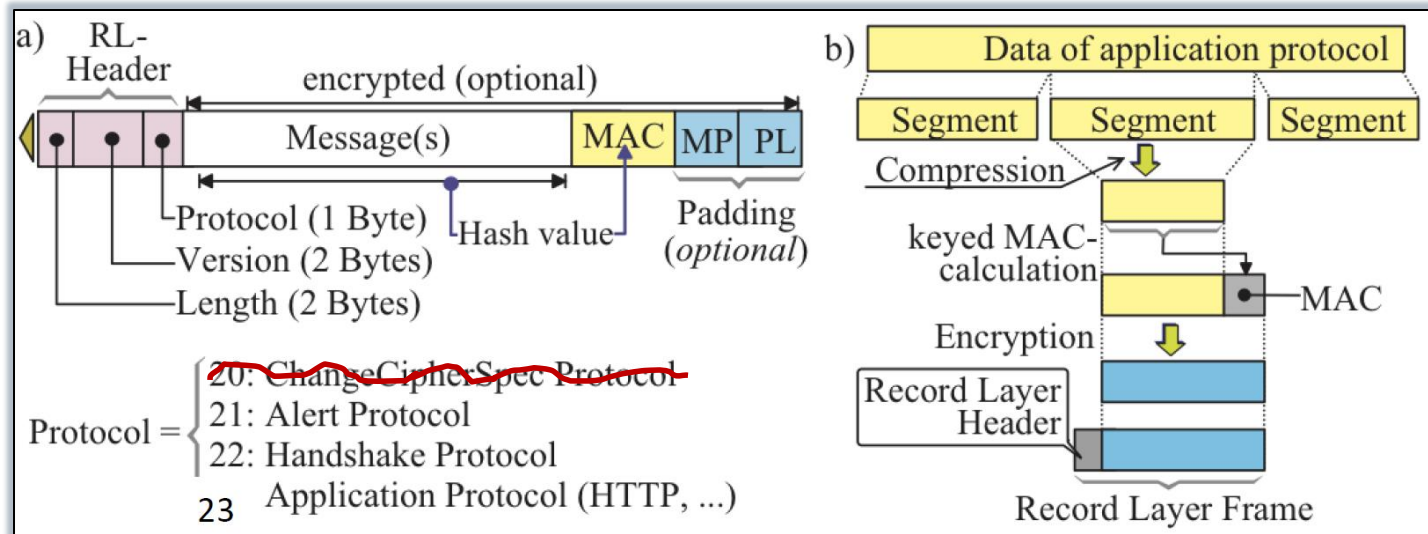
TLS 1.2: Layered Architecture



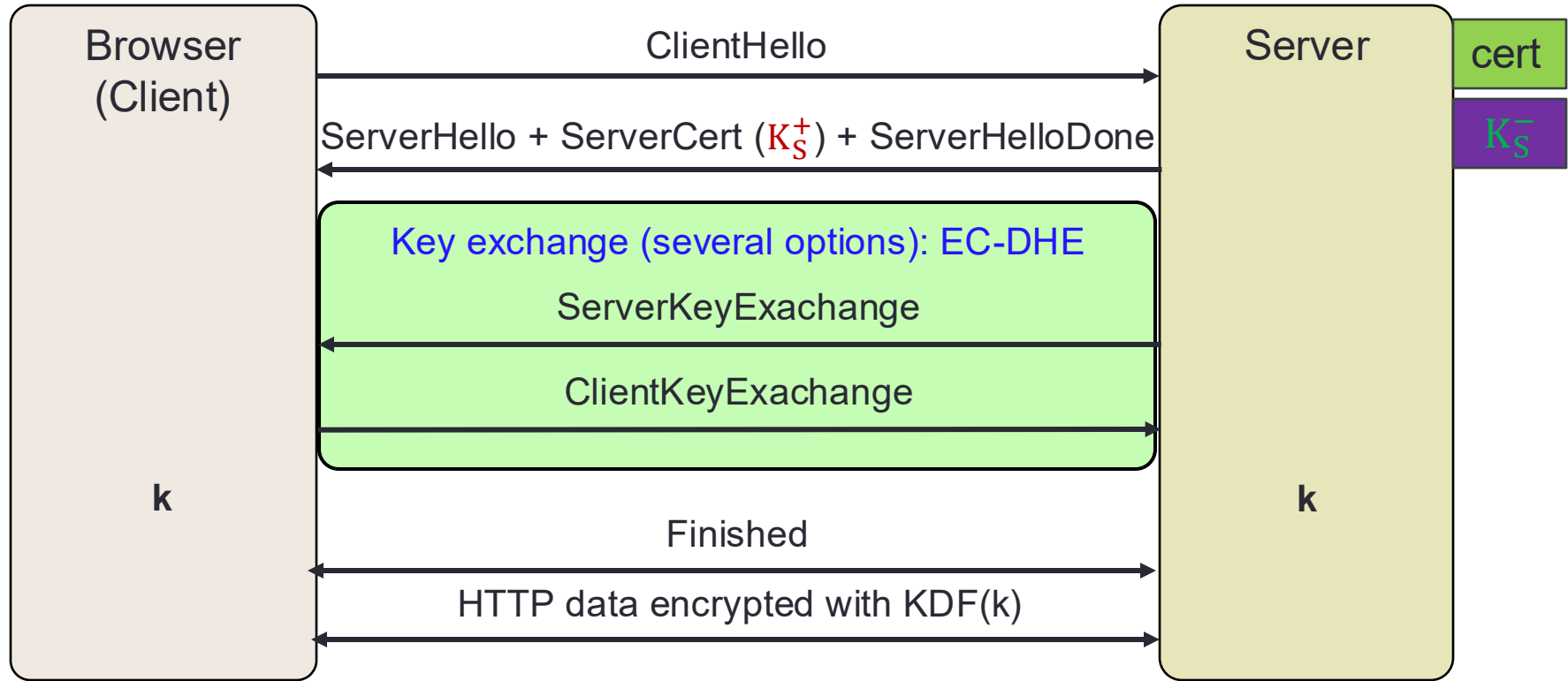
ChangeCipherSpec protocol is not supported in TLS 1.3 anymore

TLS 1.2: Record Layer

- ❑ RL is the workhorse of TLS
 - Fragment the application data into segments
 - Compression of segments
 - Integrity by adding MAC, padding (if needed), and then Encryption
 - Finally, adding required RL Header

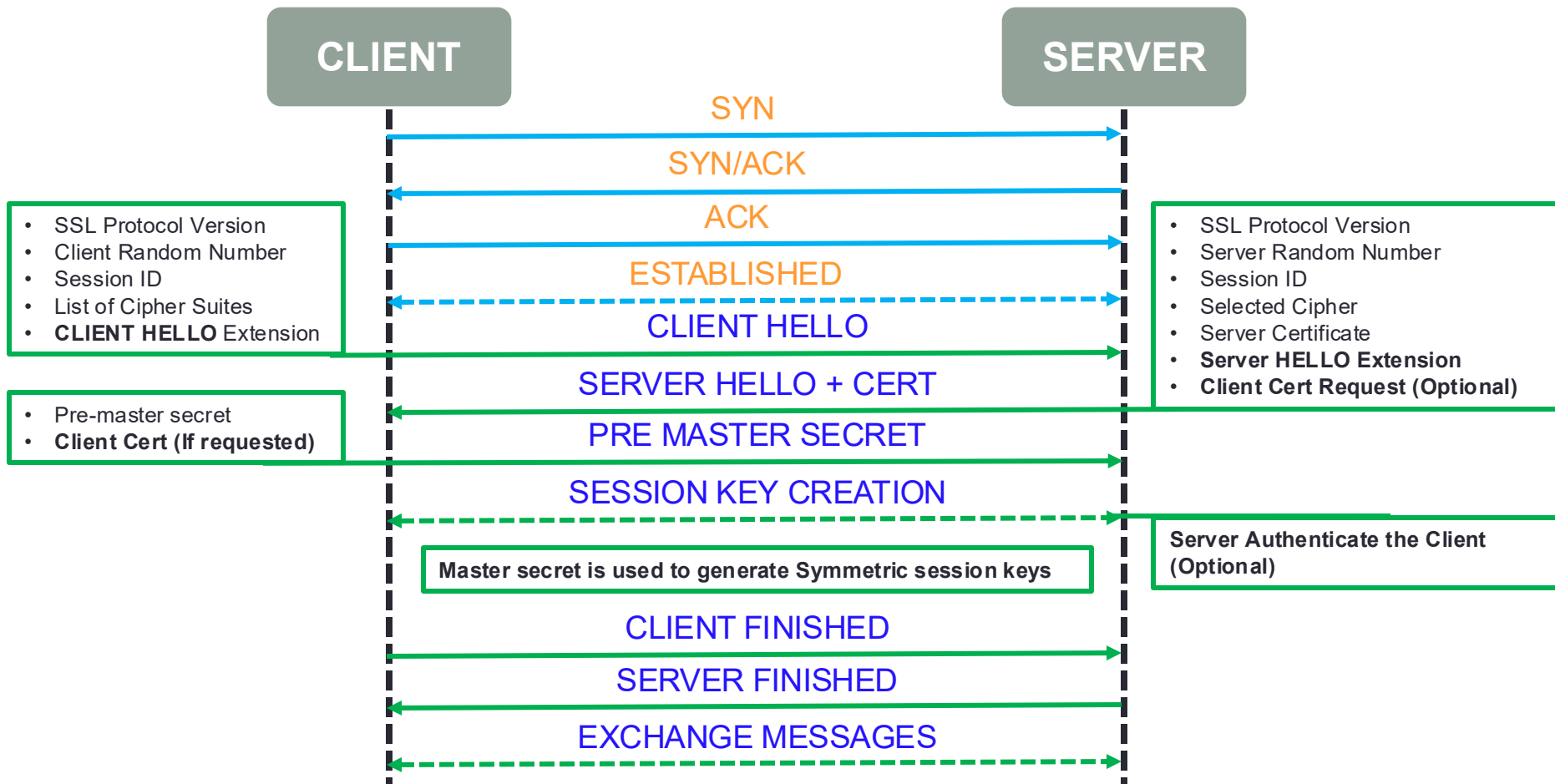


Brief Summary of SSL/TLS

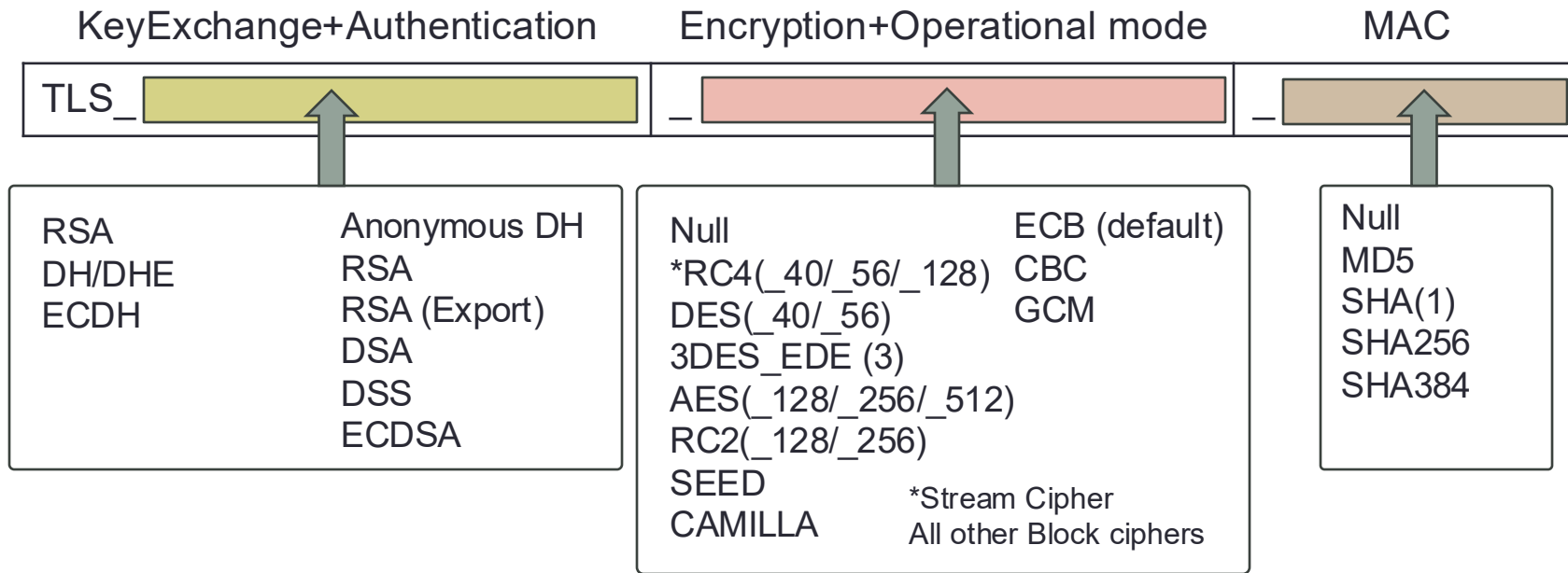


Most common: Server authentication only

Over All Communication



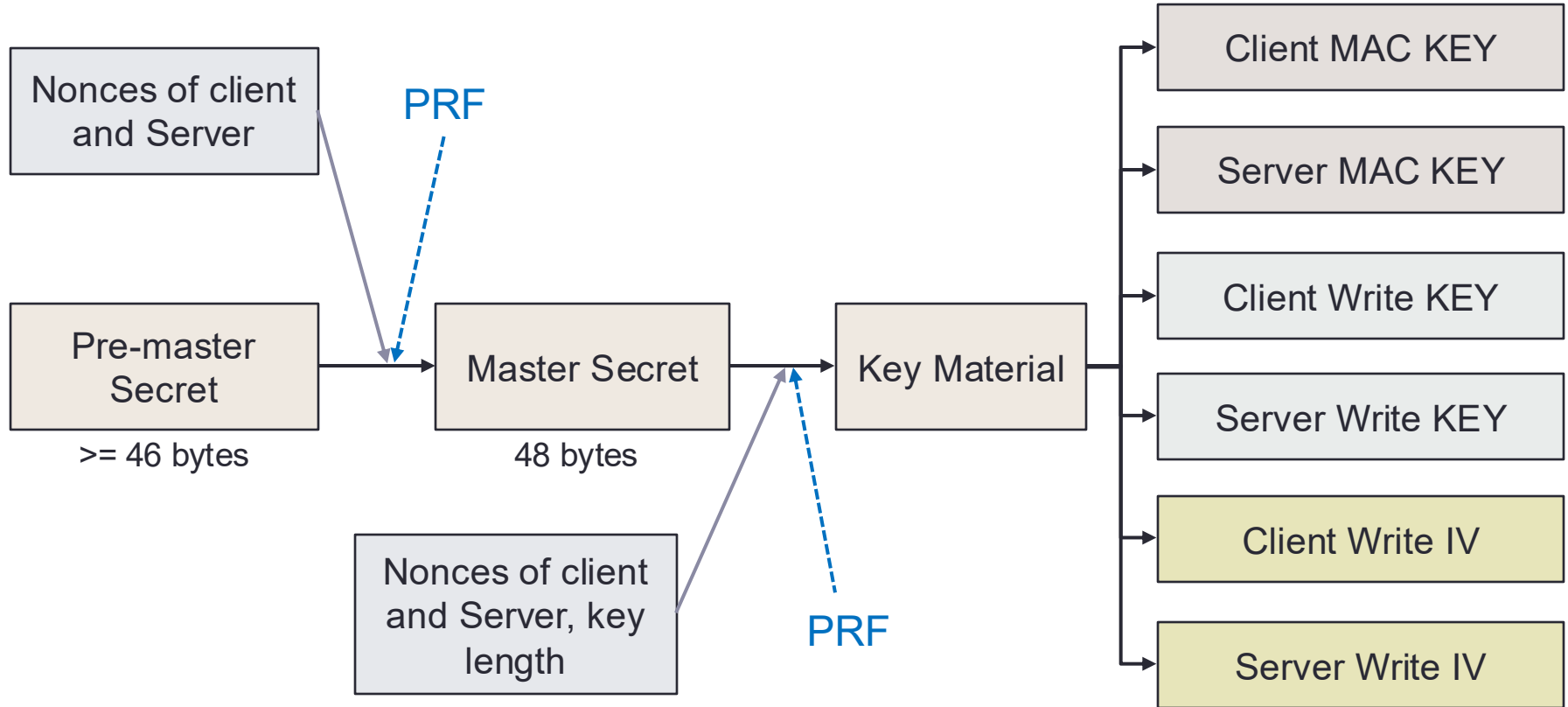
Format of Cipher Suite



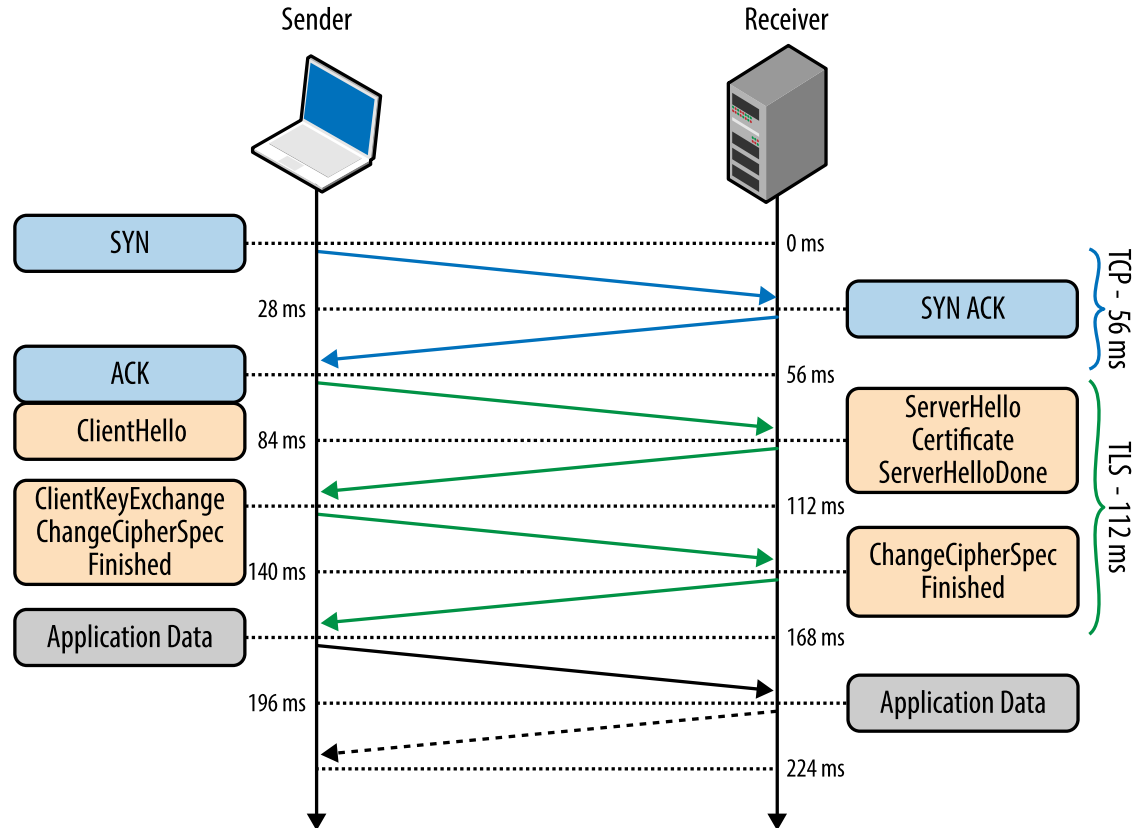
Example

- TLS_RSA_WITH_AES_256_CBC_SHA256 (TLS 1.2)
- TLS_DHE_RSA_WITH_AES_256_CBC_SHA256 (TLS 1.2)
- AEHD_AES_256_GCM_SHA384 (TLS 1.3)

Key Generation in TLS 1.2



Full TLS 1.2 Handshake with Timing Info



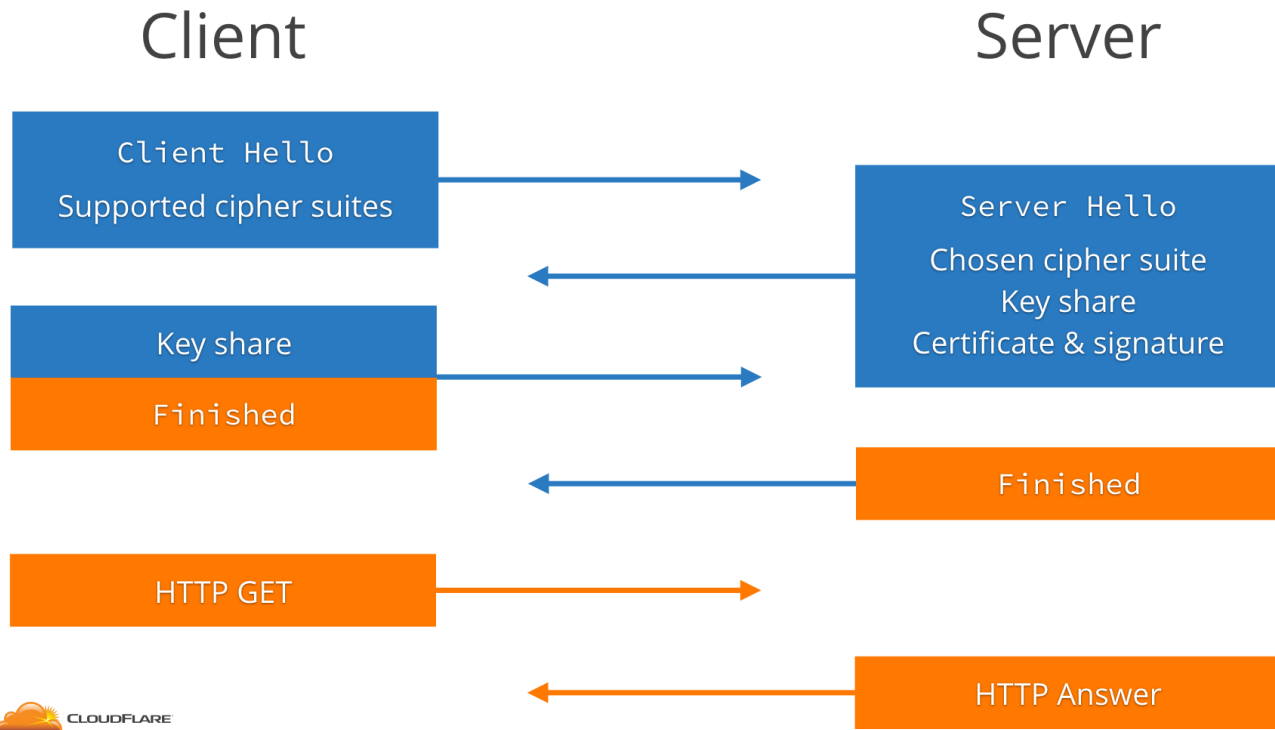
Question?

- ❑ Could Trudy launch any MITM attacks and Replay attacks by snooping into the HTTPS-TLS-TCP session established between browser (Alice) and web server (Bob)?

TLS: Guarding Against Simple Attacks

- ❑ Role of random numbers (nonces) in TLS handshake
 - Protect against connection/session replay attacks
- ❑ Role of sequence numbers in TLS session
 - Different from TCP Sequence Numbers, not added explicitly into Record Protocol Header
 - Protect against segment replay attacks
 - Protect against segment reordering or deletion by modifying TCP Sequence Numbers

TLS 1.2: ECDHE



Diffie-Hellman in TLS

❑ Fixed or Static Diffie-Hellman

- Server's public DH params like g , p and public key (g^b) are kept in Digital Cert and signed by CA
- CipherSuite: TLS_DH_RSA_WITH_AES_128_CBC_SHA256
- No Perfect Forward Secrecy (PFS)

❑ Ephemeral Diffie-Hellman

- Server and client generate fresh DH keypairs for each session
- Public DH parameters for ephemeral keypairs are signed by the private key (RSA/DSS) of Server
- Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
- Offers PFS

❑ Anonymous Diffie-Hellman

- No authentication, possible MITM attacks
- Cipher Suite: TLS_DH_anon_WITH_AES_256_CBC_SHA256

Comparison of Cipher Suits

- ❑ TLS_RSA_WITH_AES_256_CBC_SHA256 Static RSA keys for authentication and session key exchange
 - PMS is encrypted with Server's Public RSA key
 - No PFS (Perfect Forward Secrecy)
 - No Server Key Exchange Msg in TLS handshake
- ❑ TLS_DHE_RSA_WITH_AES_256_CBC_SHA256
 - Static RSA keys for authenticating Server's Certificate
 - DH with ephemeral key pairs for session key exchange
 - Server Key Exchange Msg in TLS handshake carries public DH parameters
 - PMS (g^{ab}) is never exchanged, but locally derived by both!
 - Offers PFS

Note: Static RSA keys for authentication and ephemeral RSA keys for key exchange offer PFS but never used as DHE/ECDHE are more efficient

TLS Vulnerabilities

Classification of TLS Vulnerabilities

I) Conceptual flaws in TLS and the resulting exploits

- ❑ Protocol downgrades, connection renegotiation, session resumption, incomplete/vague specs
 - 3SHAKE, TLS Renego MITM attacks, POODLE, LOGJAM, FREAK

II) Vulnerabilities due to using weak crypto primitives

- ❑ Block ciphers that operate in CBC mode
 - Sweet32, ROBOT, Lucky13

III) Implementation vulnerabilities

- ❑ Faulty implementations gave rise to cross-layer protocol attacks and/or side channel attacks
 - BEAST, CRIME, TIME, BREACH, HEIST, SLOTH, DROWN
 - SMACK, ROCA, HeartBleed

SSL/TLS Attacks

- ❑ Heartbleed attack on Heartbeat extension
- ❑ TLS DoS/DDoS attacks
- ❑ POODLE (Padding Oracle On Downgraded Legacy Encryption)
- ❑ FREAK: A Downgrade attack
- ❑ TLS Renegotiation MITM attacks
- ❑ Replay attacks

Heartbleed

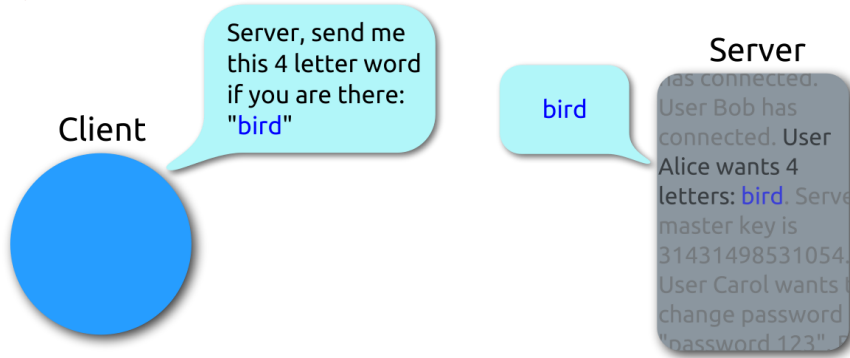
Heartbeat Extension

- ❑ DTLS/TLS: Heartbeats to keep the connection alive without continuous data transfer and renegotiations (liveness check)
- ❑ Liveness check at a much higher rate than TCP keep-alive feature
 - Multiples of RTT (1 sec to several minutes)
- ❑ DTLS uses it also for path MTU (PMTU) discovery
- ❑ Modes: `peer_allowed_to_send` / `peer_not_allowed_to_send` in `Client_Hello` message
- ❑ Alice: `heartbeat_request` and its echo (`heartbeat_response`) by Bob
- ❑ `heartbeat_request`: `msgType`, `payloadLen`, `payload`, `padding`

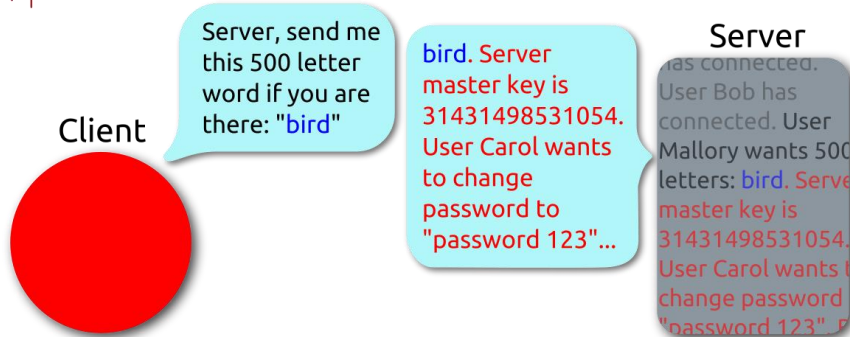
Heartbeat Message



Heartbeat – Normal usage



Heartbeat – Malicious usage



Credit: Fenix Feather

- ❑ Purpose: Proving to other party that connection is still alive by sending keep-alive message
 - Heartbeat extension ([RFC 6520](#))
- ❑ It includes message length
- ❑ Password/key leaking security bug in OpenSSL
- ❑ In 2014, affected 17% of SSL servers

Is it design flaw in TLS??

SSL Heartbleed Attack



The coding mistake in OpenSSL that caused Heartbleed!

`memcpy(bp, pl, payload);`

CVE-2014-0160 (Buffer Overread Bug)

Vulnerable version: OpenSSL 1.0.1 through 1.0.1f

Prevention: Update to the latest version of OpenSSL and if that is not possible, recompile the already installed version with `-DOPENSSL_NO_HEARTBEATS`

DoS/DDoS

SSL/TLS Dos/DDoS Attack

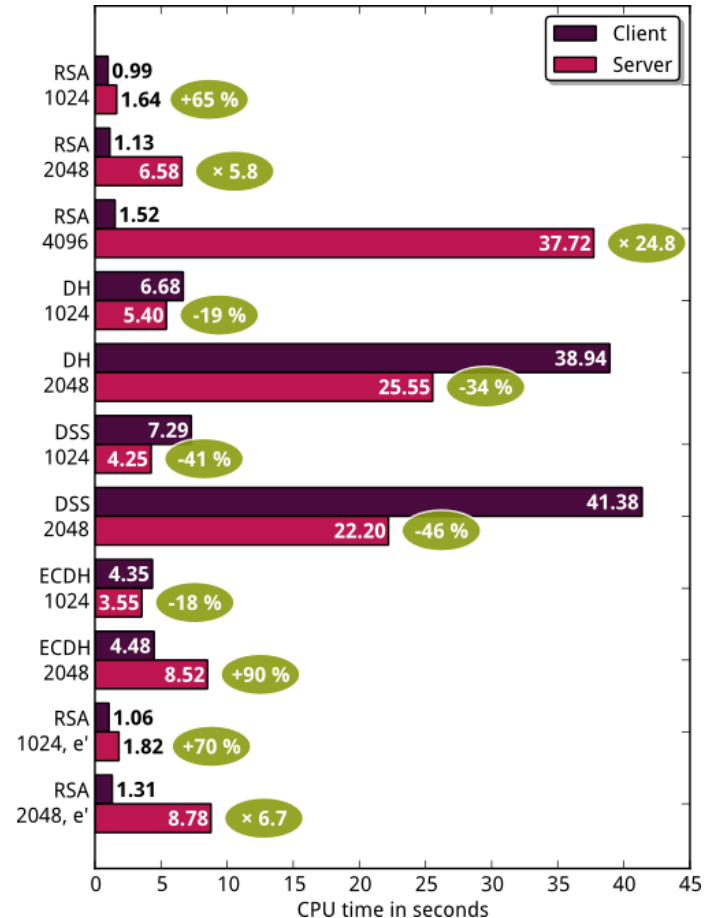
□ HTTPS Floods

□ Launching many TLS sessions per sec

- (Bogus) SSL handshake messages consume more resources (15x) at Server than at client (attacker)

- Client encrypts pre-master-secret (PMS) which server has to decrypt in RSA based keys

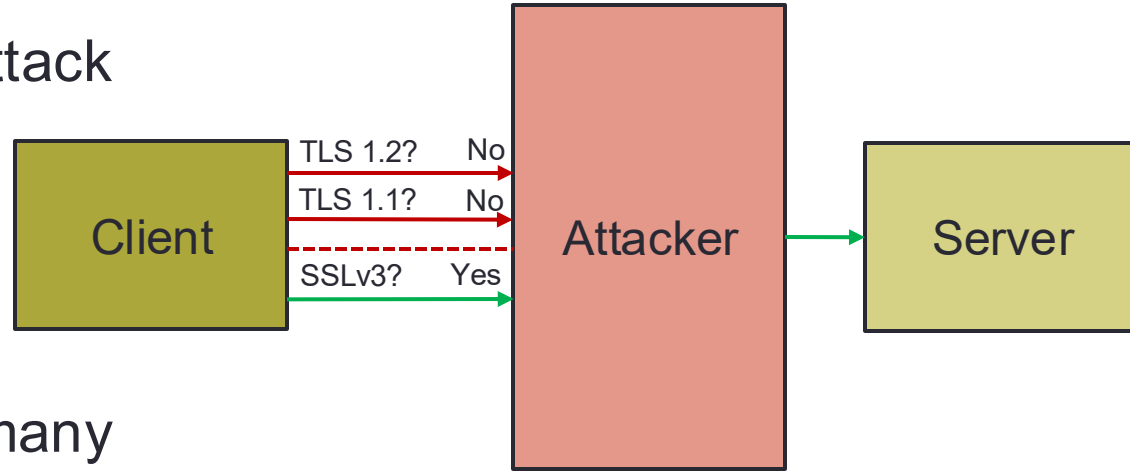
□ Solution: Rate limit TLS handshakes per source IP address at server



POODLE

POODLE Attack

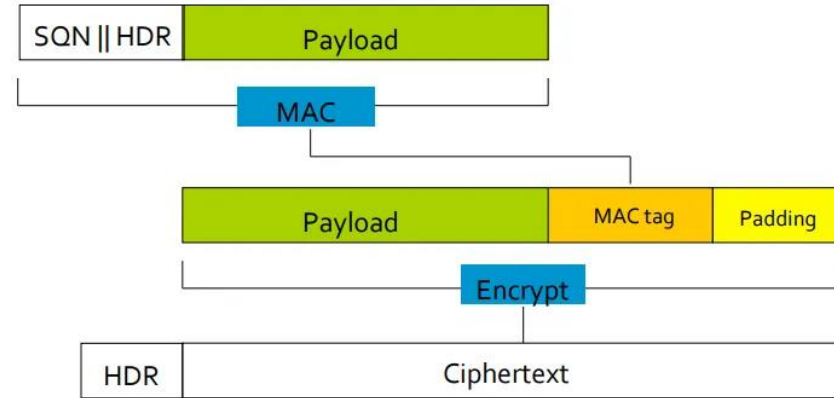
- ❑ Complex, cross-layer attack
- ❑ Padding Oracle On Downgraded Legacy Encryption (POODLE)
- ❑ Exploited the fact that many servers still supported SSLv3 and suffer from a CBC vulnerability



<https://github.com/thomaspatzke/POODLEAttack>
[What is the POODLE Attack? | Acunetix](#)

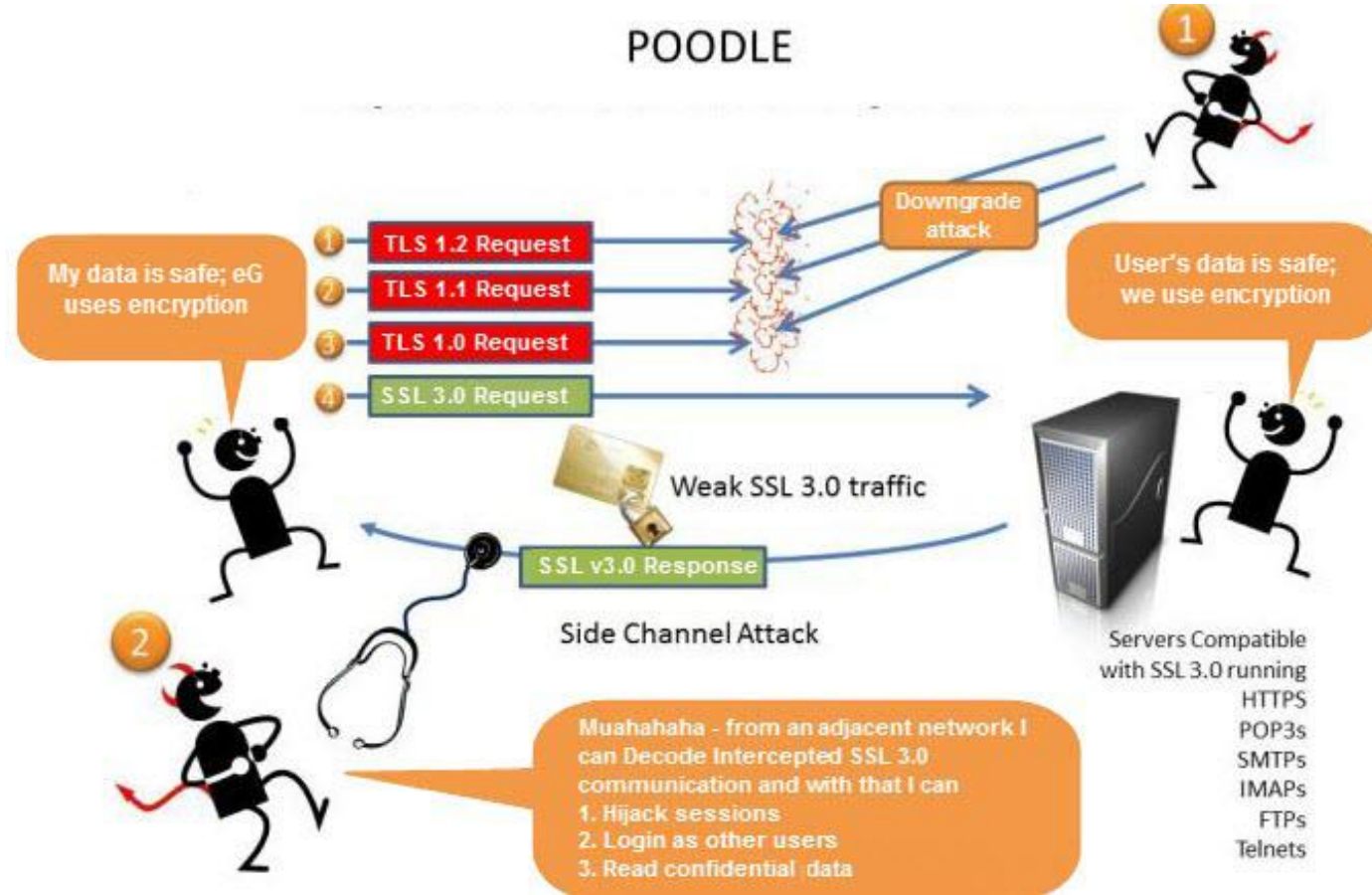
MAC, Paddings then Encrypt in SSLv3

- ❑ Block Cipher needs data in multiple of block size
 - If block size is of 8 byte, data must be multiple of 8
 - If it is not, padded it with unimportant data
- ❑ SSL requires, last byte of last block must contain length padding length
 - Possible values are 00 to 07
 - If the data is of multiple block size, add extra block with only padding
- ❑ Padding Oracle
 - Attacker know or can guess by data is rejected by the server
 - Two possibility, either MAC was wrong or padding is incorrect



POODLE

POODLE



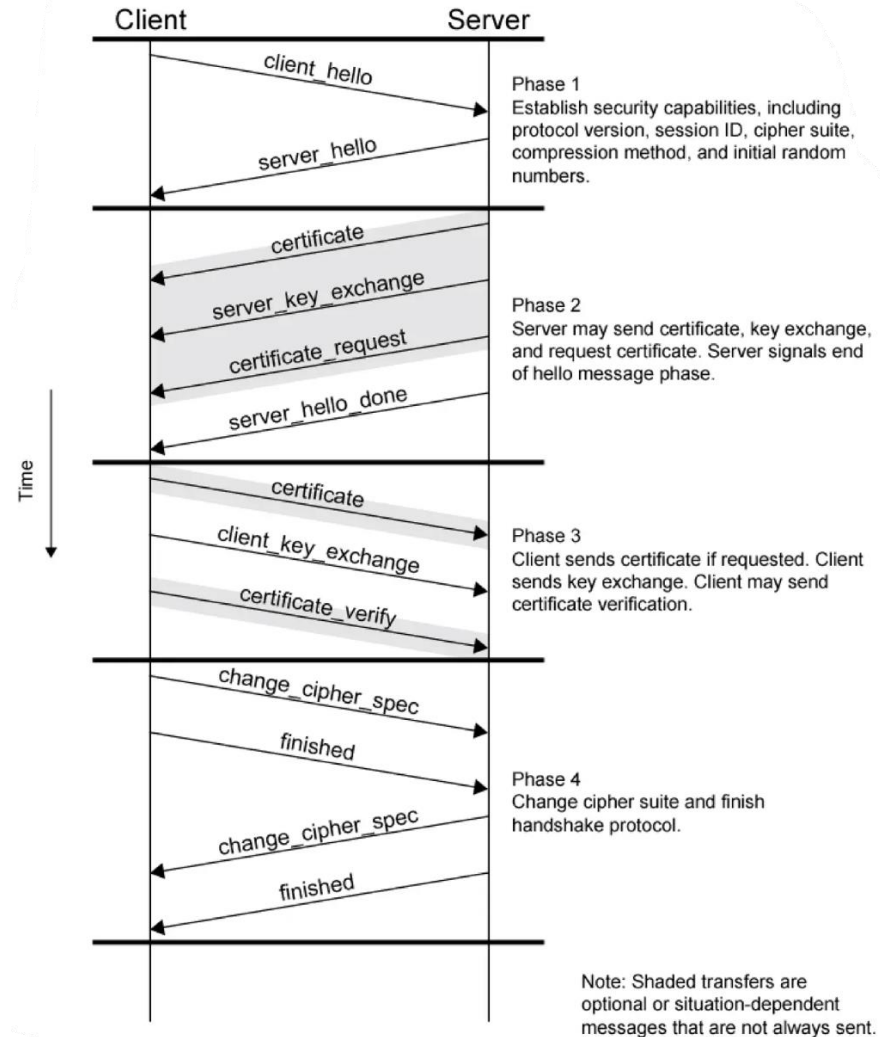
© 2014 Critical Watch

Mitigation: Configure the server to use only TLS 1.2 and no older protocol

FREAK

Four Phases of TLS Handshake

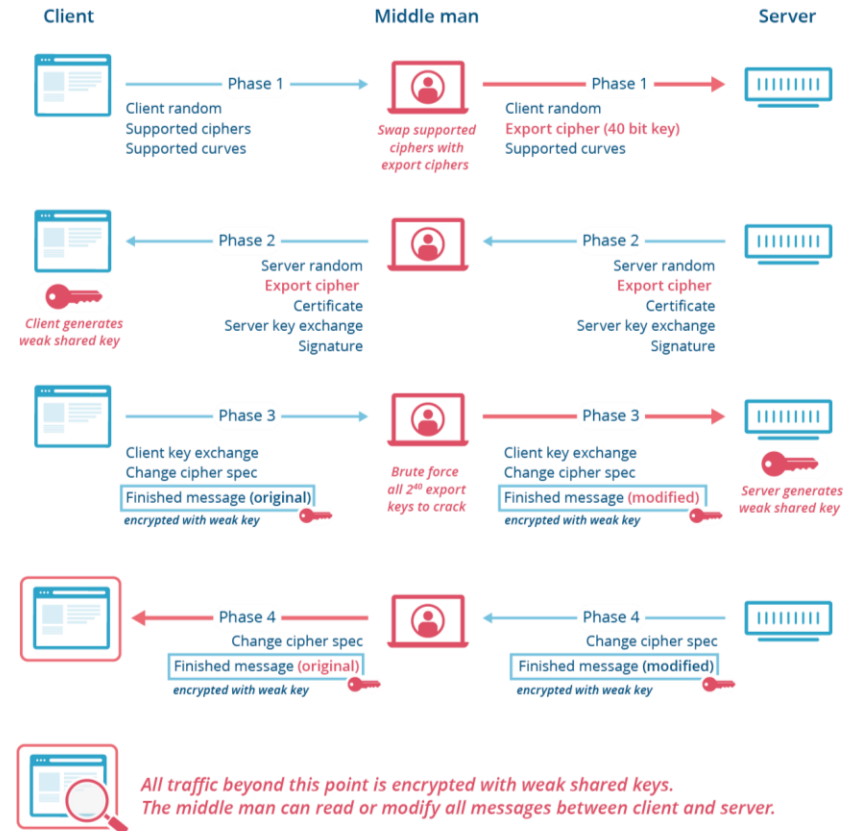
- ❑ **Phase-1:** Both ends agree upon Cipher Suite
- ❑ **Phase-2:** Server sends its digital Cert signed by a CA
- ❑ **Phase-3:**
 - Client sends a pre-master secret (PMS) key encrypted with Server's public key
 - Client may also send a signed hash of all of its previous messages in Cert_Verifymsg
- ❑ **Phase-4:** Handshake is completed and a secure connection is established 11



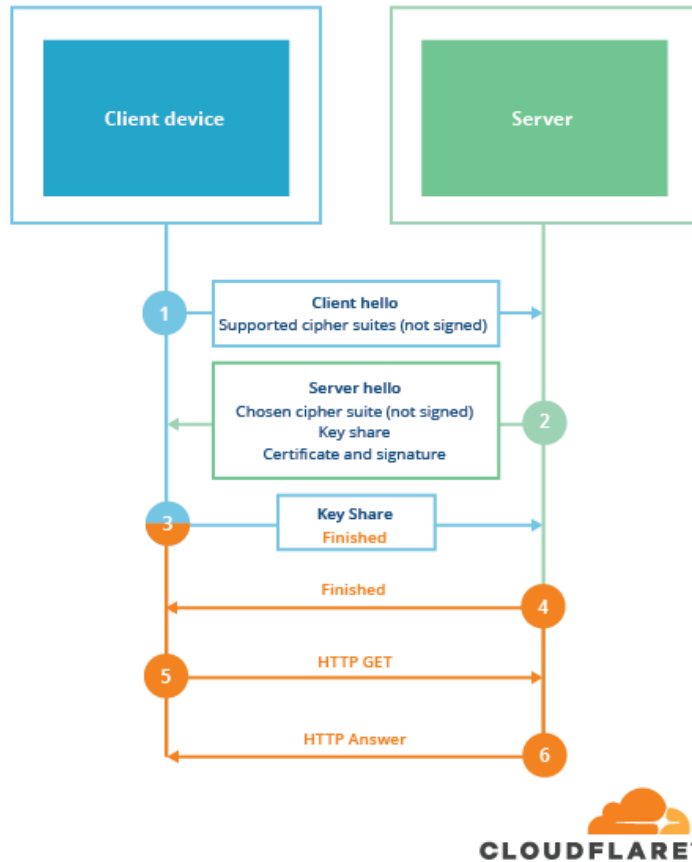
Downgrade Attack (FREAK)

- ❑ FREAK, LogJam & CurveSwap attacks took advantage of two things:
 - Support for weak ciphers in early TLS versions
 - Part of handshake which is used to negotiate which cipher to use is not signed
- ❑ **Note:** In Phase-4: Attacker has to modify FINISHED msg before sending it to Client as it includes hash of all handshake msgs exchanged in both the directions.

Downgrade Attack (FREAK)



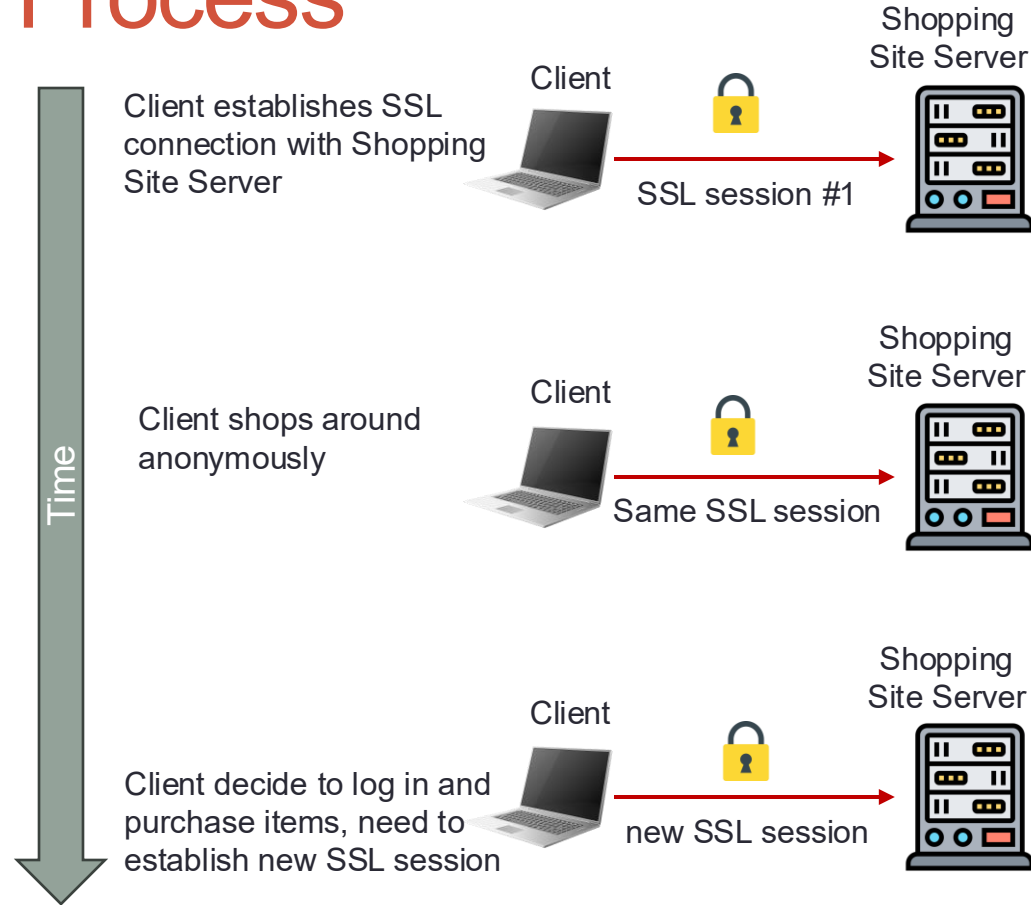
TLS 1.2 ECDHE



RENEGOTIATION

TLS Renegotiation Process

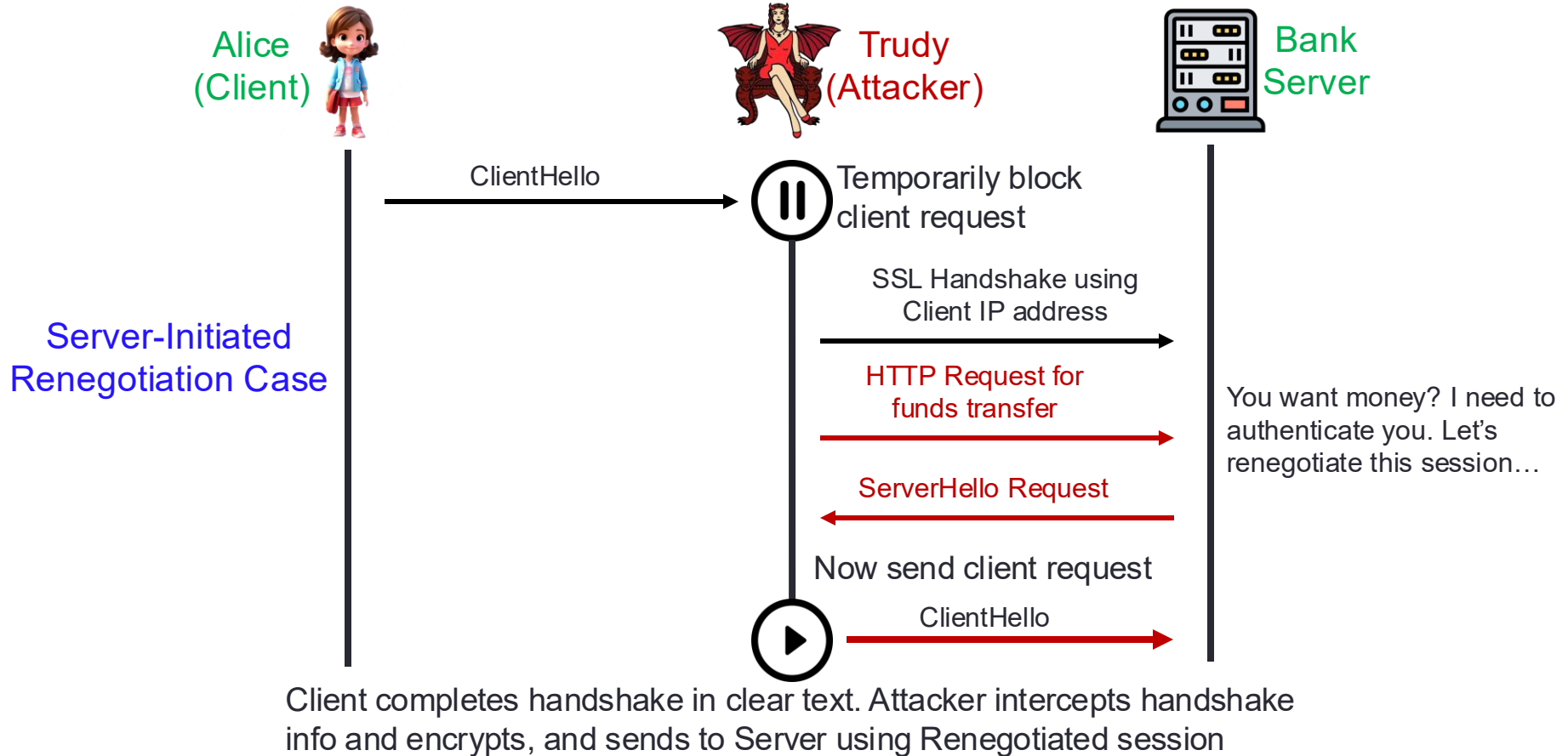
- ❑ TLS 1.2 allows either the client or the server to initiate renegotiation --a new full handshake on encrypted channel to establish new cryptographic parameters
- ❑ Eg: HTTP servers support renegotiation to request client certs for a protected resource
- ❑ TLS renegotiation messages (including types of ciphers and key material) are encrypted and then sent over the existing SSL connection (encrypted)
- ❑ Unfortunately, although the new handshake is carried out using the cryptographic parameters established by the original handshake, **there is no cryptographic binding between the two.**



TLS Renegotiation Attacks

- ❑ Discovered by Marsh Ray and Steve of PhoneFactor(2009)
- ❑ Assumption: client-initiated renegotiation supported at Server
- ❑ **DoS attack: One TCP session which is performing several expensive crypto operations**
 - A single server can perform between 150-300 handshakes per second while a single client can request around 1000 handshakes per second!
 - Detection is hard because fewer TCP sessions unlike TCP SYN Flood attack and fewer visible TLS handshakes unlike TLS DoS attacks
 - Web servers can limit number of renegotiations per client or do not allow client-initiated renegotiations

TLS Renegotiation Based MITM Attack



TLS Renegotiation Attacks

❑ Mitigation techniques

- Disable TLS renegotiation especially from clients
 - Apache does not allow client-side renegotiation
- `openssl s_client` can be used to test if TLS renegotiation is really disabled. Sending `R` on an empty line triggers renegotiation.
 - `$ openssl s_client -connect www.google.com:443 -tls1`
- Rate limiting of TLS handshakes
- Increase server-side processing (scaling on-demand)
- IETF's solution in [RFC5746](#)
 - Renegotiation Info Extension in Hello messages to cryptographically tie renegotiations to the TLS connections they are being performed over, thus preventing this attack
 - `client_verify_data`(content of FINISH msg) of initial handshake

TLS 1.3

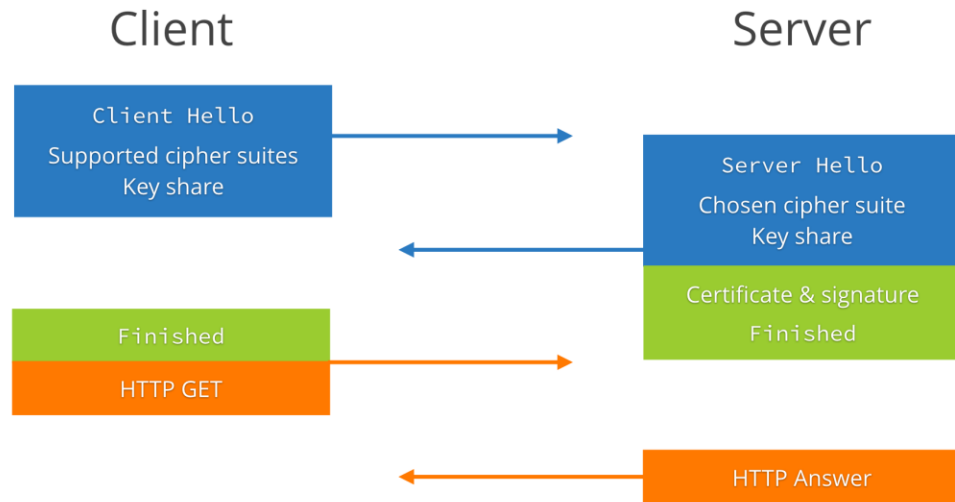
TLS 1.3 (RFC 8446)

- ❑ Much faster and more secure compared to TLS 1.2
 - 1-RTT for connection setup (vs 2-RTT in TLS 1.2)
 - 0-RTT for resumption (vs 1-RTT in TLS 1.2) for some type of application data
- ❑ More of the handshake is **encrypted** and **full handshake is signed**
- ❑ Downgrade protection
- ❑ AEAD cipher suites (which compute MAC & Encrypt at one shot) instead of MAC-then-Encrypt in TLS 1.2
- ❑ Phased out insecure protocols, ciphers & algorithms present in TLS 1.2 while bringing in secure replacements
- ❑ Some of **deprecated** features in TLS 1.3
 - RC4 stream cipher, CBC mode ciphers (POODLE, Lucky 13)
 - Export-strength ciphers (FREAK)
 - RSA key transport (Only (EC)DHE, PSK-only and hybrid key agreement modes for ensuring forward secrecy) (DROWN)
 - Various Diffie-Hellman groups (Note: Restricted DH parameters to ones that are known to be secure)
 - SHA-1, MD5 (SLOTH)
 - DES, 3DES
 - Compression (CRIME)
 - Renegotiation

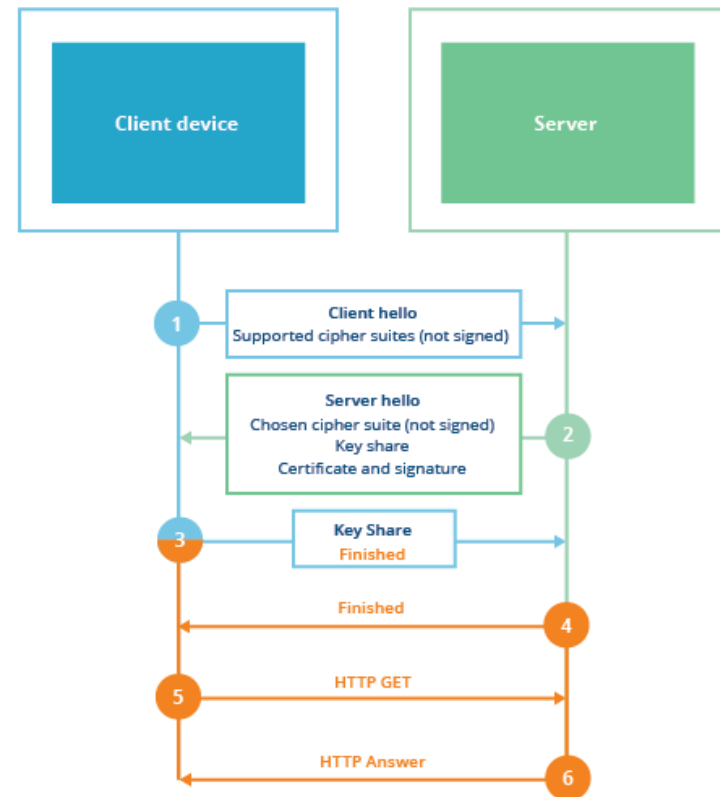
TLS 1.3 Cipher Suites

- ❑ Cipher suites are simplified a lot!
- ❑ Do not specify the certificate type (e.g., RSA/DSA/ECDSA) or the key exchange mechanism (e.g., DHE or ECDHE)
 - DHE and ECDHE
 - Pre-shared key (PSK)
 - PSK /w (EC)DHE
- ❑ Digital Signature (Authentication) algorithms
 - RSA (PKCS#1 variants)
 - ECDSA
- ❑ Only 5 Cipher Suites:
 - TLS_AES_256_GCM_SHA384
 - TLS_CHACHA20_POLY1305_SHA256
 - TLS_AES_128_GCM_SHA256
 - TLS_AES_128_CCM_8_SHA256
 - TLS_AES_128_CCM_SHA256

TLS 1.3 (ECDHE)

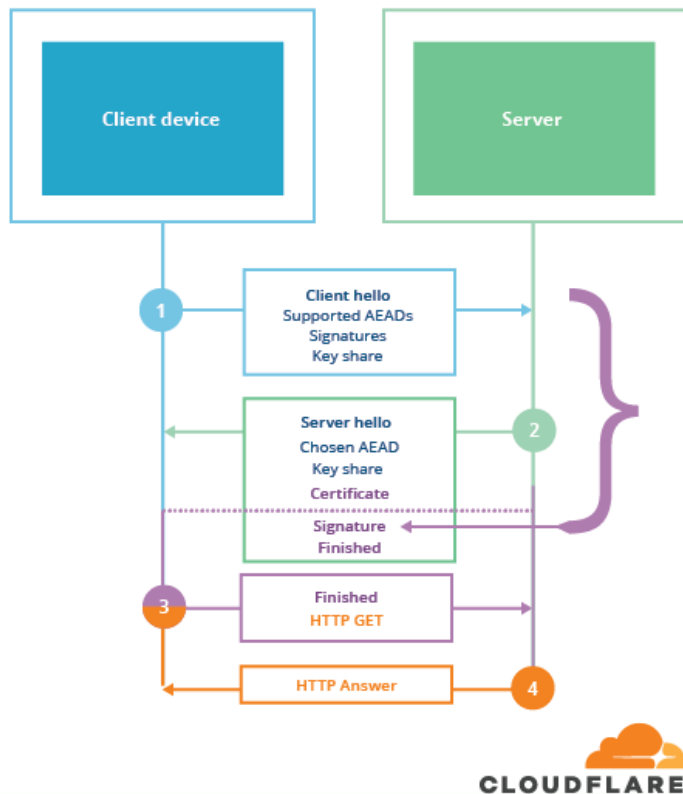


TLS 1.2 ECDHE

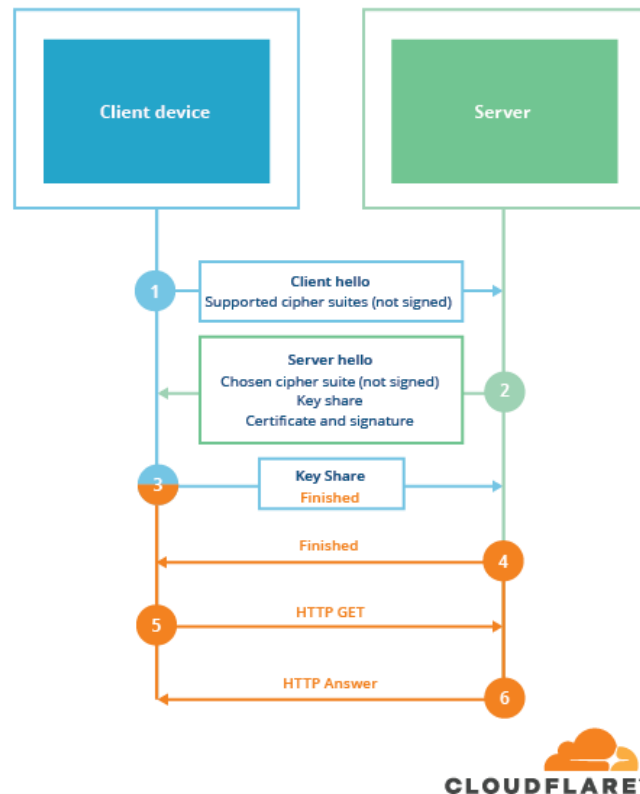


HelloRetryRequest instead of ServerHello, if Client sends unacceptable (EC)DHE groups

TLS 1.3

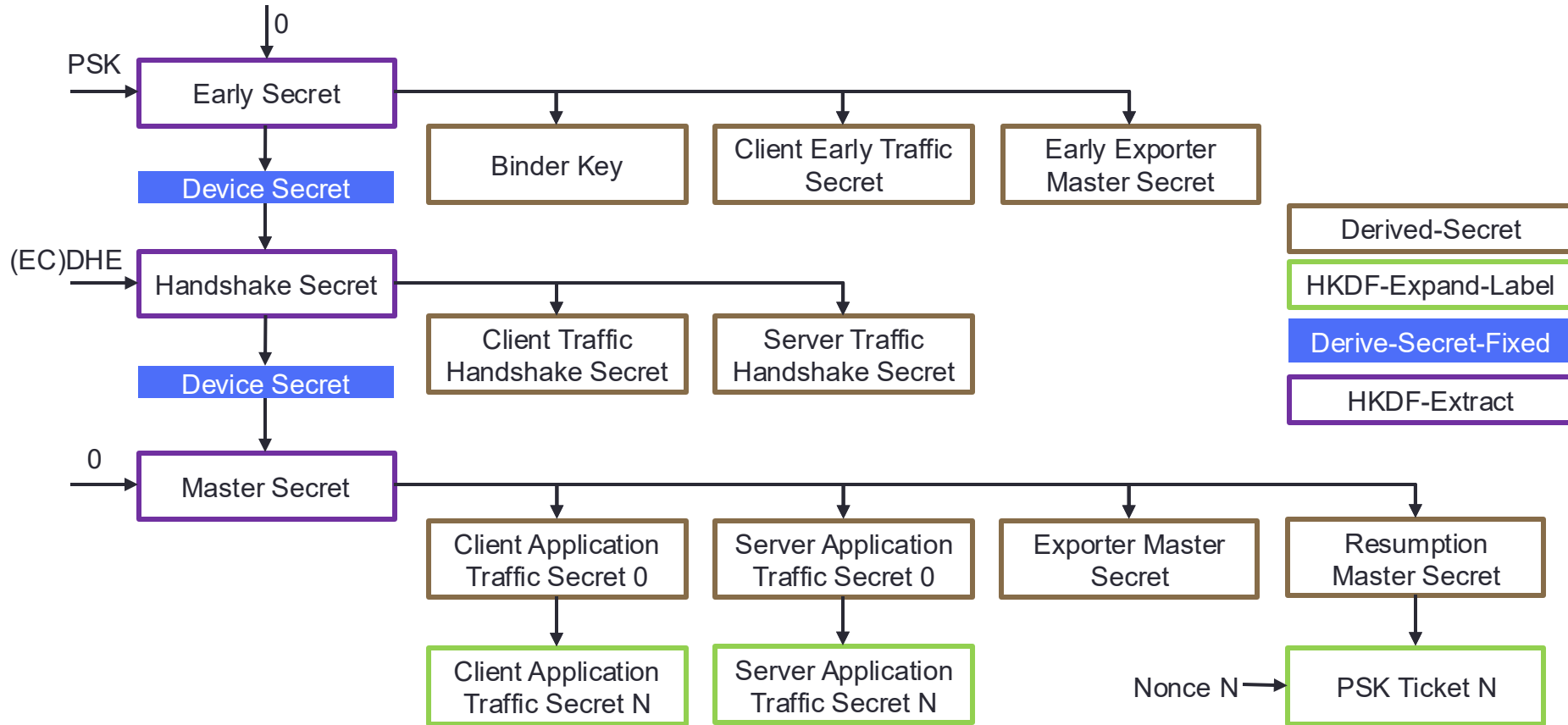


TLS 1.2 ECDHE

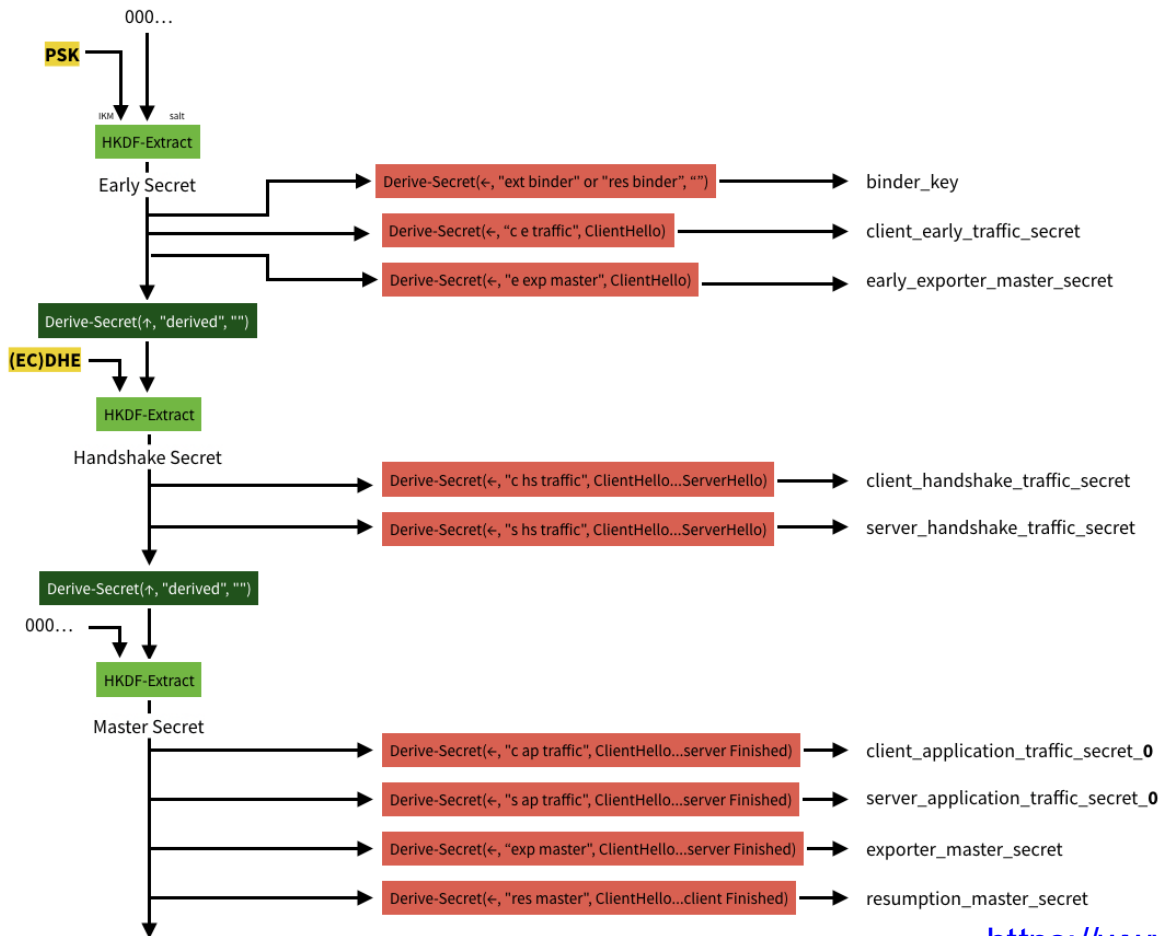


Signature Extension in server to client: All previous message are signed with RSA private key of Server to ensure no tampering in TLS 1.3

Key Schedule Generation in TLS 1.3



Key Schedule Generation in TLS 1.3...

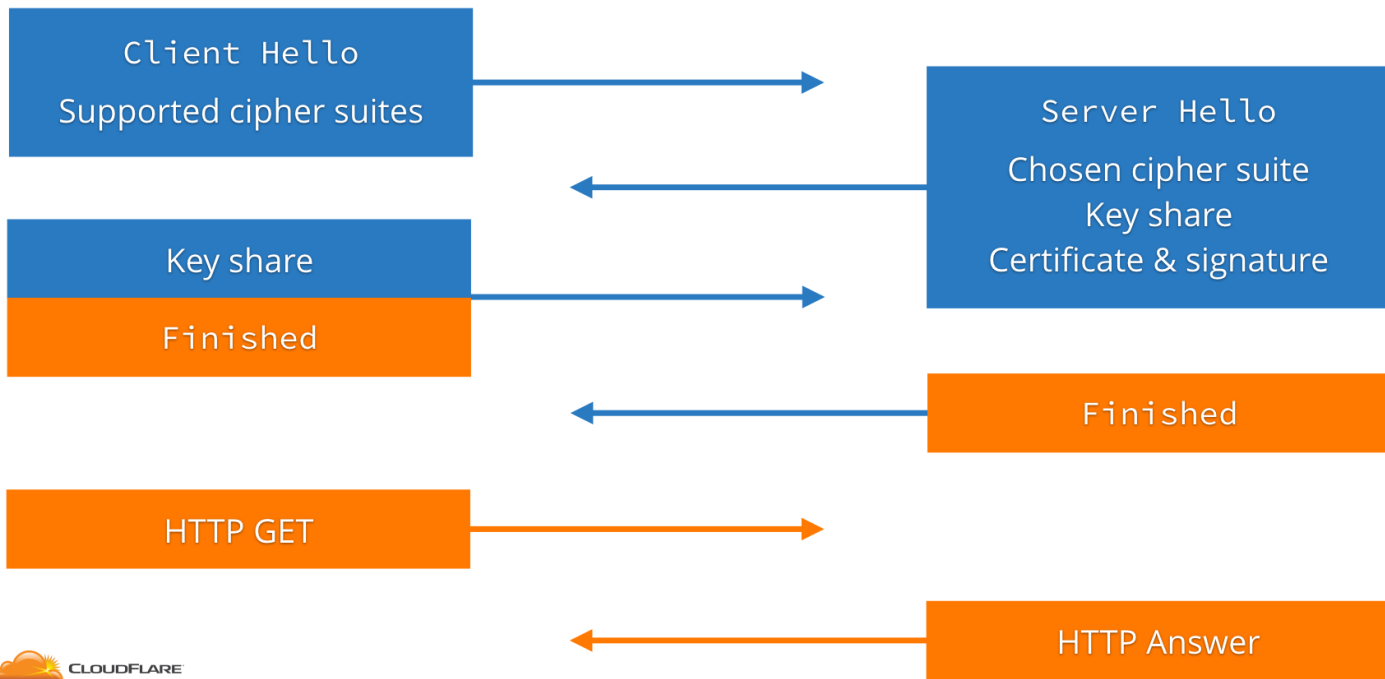


Session Resumption

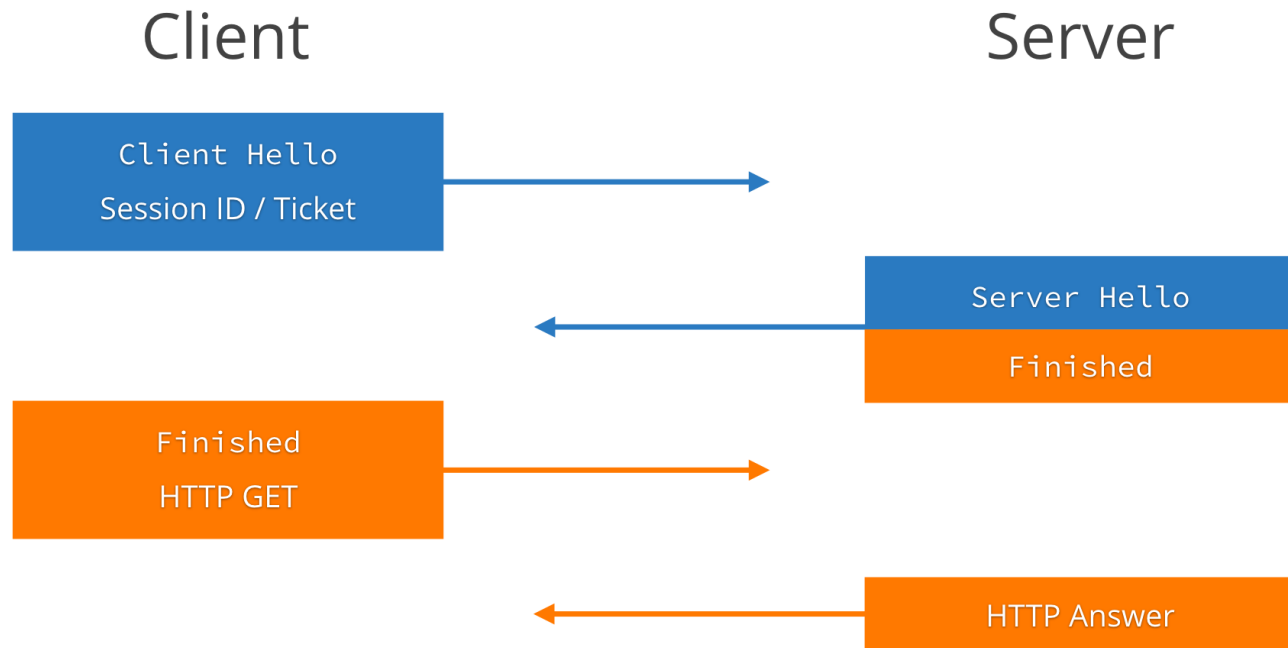
TLS 1.2: ECDHE Handshake (2-RTT)

Client

Server



TLS 1.2: Session Resumption (1-RTT)



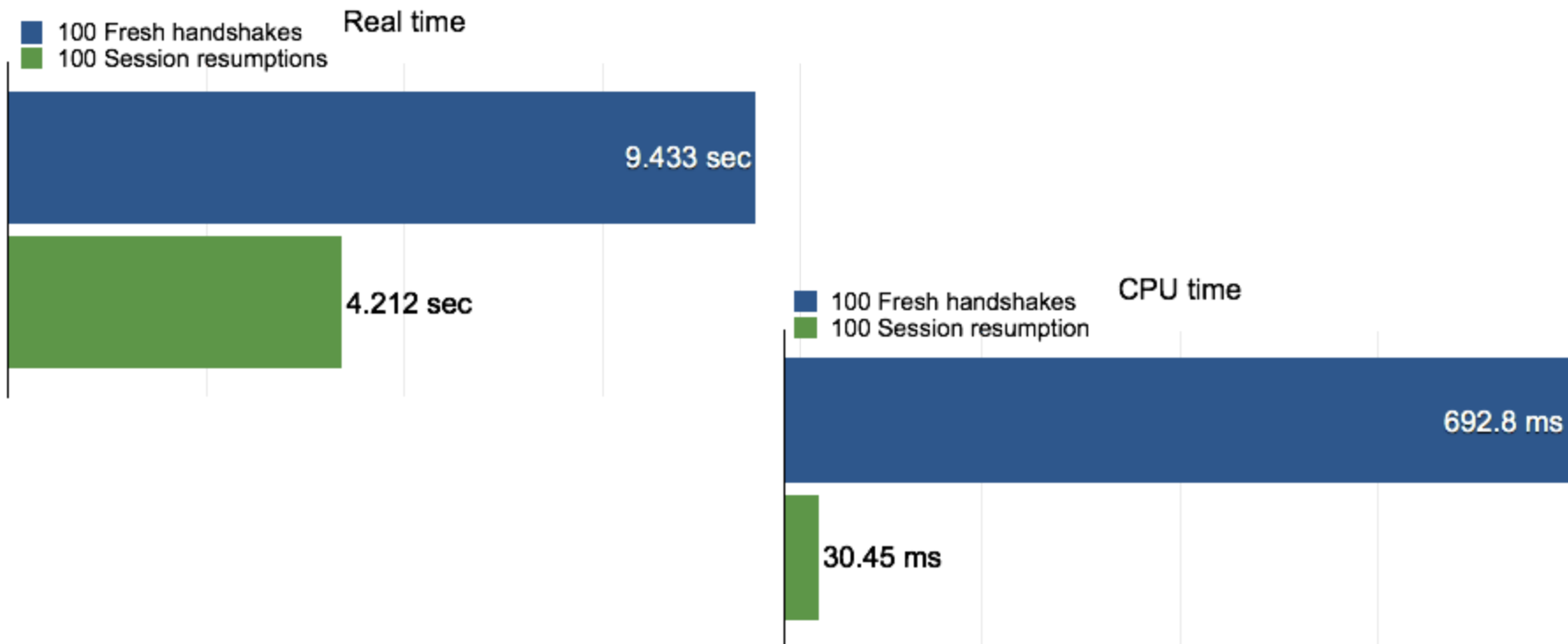
Session Resumption in TLS 1.2

- ❑ Helps to speed up the handshake process
- ❑ Do not confuse Session Resumption with Session Renegotiation
 - Renegotiation uses the same TLS/TCP connection to renegotiate security parameters which does not involve Session ID or Session Tickets

1) Stateful Resumption (Server side, Session ID)

- Server saves state of the previous session, in particular the **KeyMaterial** derived, in a db/cache indexed with Session ID
- Stored MasterSecret is used to generate **session keys** together with new random variables for the new session

Performance of Session ID Resumption



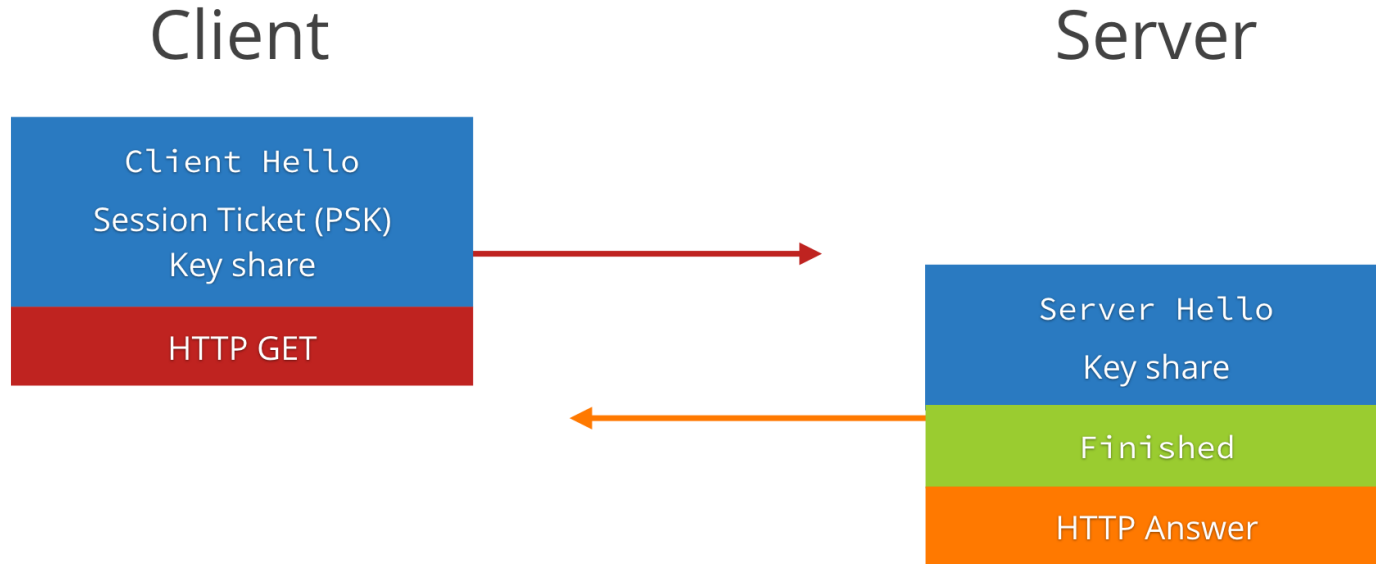
<https://blog.cloudflare.com/tls-session-resumption-full-speed-and-secure/>

Session Resumption in TLS 1.2

2) Stateless Resumption (Client side, Session Ticket) [RFC5077](#)

- Server sends Session Ticket to client in the previous session
 - Improves scalability of servers
 - Helps to load balance requests (TLS sessions) across servers
 - Possible in case of Session ID based resumption?
 - Ticket includes encrypted state <ciphersuite, MasterSecret, and Timestamp (for checking validity of ticket), et> and MAC
 - Encrypted using a **unique session ticket enc key (STEK) known only to the server** and MAC protected (HMAC-SHA-256)
 - NewSessionTicket handshake message from the server contains session ticket and ticket_lifetime_hint (not encrypted)
 - A server may treat a ticket as valid for a shorter or longer period of time than what is given in the ticket_lifetime_hint 😊
 - Protection against Stolen tickets, Forged tickets and DoS attacks by choosing strong encryption/MAC algos

TLS 1.3: Resumption (0-RTT Data)



Note: Session Ticket = PSK, encrypted by Server with session ticket encryption key (STEK) known only to the server

Issues With 0-RTT Resumption

❑ Lack of full forward secrecy

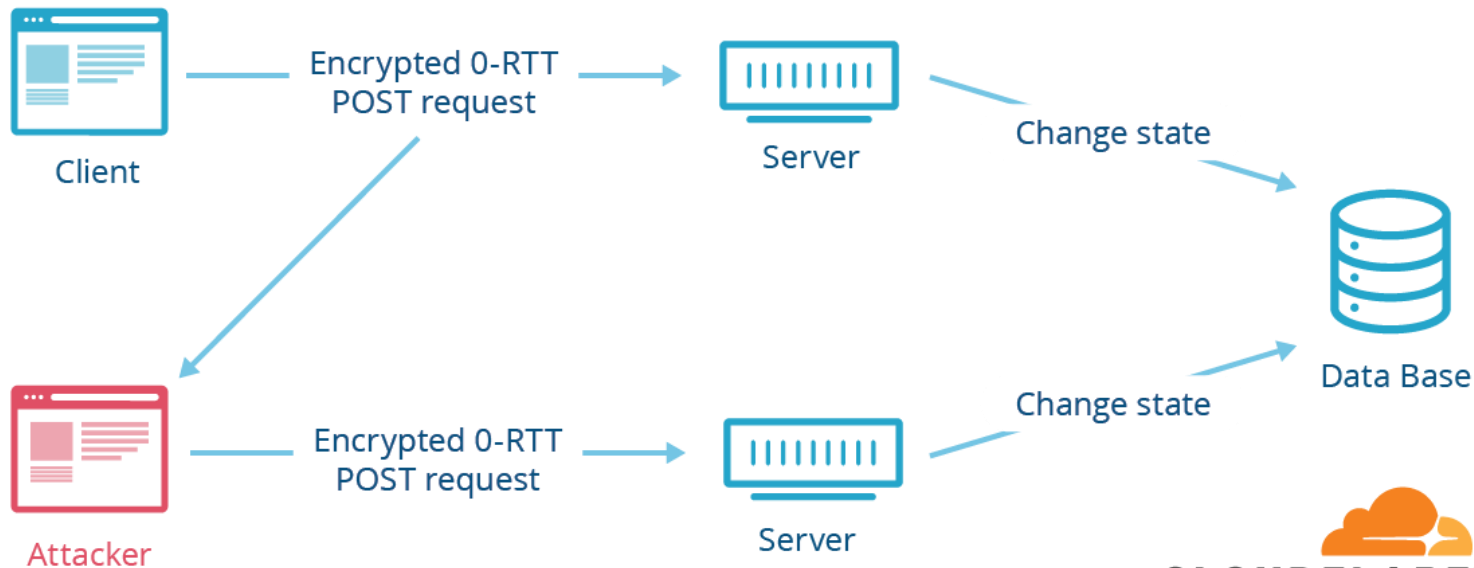
- If **session ticket encryption keys (STEK) are compromised**, an attacker can decrypt 0-RTT data (encrypted using PSK as salt to HKDF-Extract function) sent by the client on the first flight (**but not the rest of the data in session**)
- **Solution:** Rotating STEKs regularly (e.g., weekly)

❑ Replay attacks

- Attacker replays 0-RTT data (e.g., HTTP GET/POST)
- POST is not replay-safe
- **Solutions:**
 - 1) Don't allow 0-RTT resumption on POST and allow resumptions only for some period on "safe" requests (GET) and
 - 2) Each session ticket is valid only once

Replay Attacks on TLS 1.3

0-RTT Attack



<https://blog.cloudflare.com/introducing-0-rtt/>

Backward Compatibility & Extensibility

- ❑ Version in Record Header is set as "3,1" (TLS 1.0) instead of "3,4" (TLS 1.3 for interoperability with earlier implementations)
- ❑ Even for TLS 1.3, client protocol version is hardcoded to "3,3" (TLS 1.2) in Handshake messages for backward compatibility with middleboxes and legacy servers!
 - Version negotiation is performed using the "Supported Versions" extension in client & server hello messages
- ❑ A number of TLS protocol values, referred to as GREASE (Generate Random Extensions And Sustain Extensibility) values, are reserved
 - A client **MAY** select one or more GREASE cipher suite values at random and advertise them in the "cipher_suites" field
 - Correctly implemented server will ignore these values like any unknown values so that **new capabilities may be introduced while maintaining interoperability**